Inserção adaptativa de elementos de interface em malhas 2D

Operadores topológicos sobre estruturas de dados compactas baseadas em índices

Jeferson Coelho
Tecgraf/PUC-Rio*

Relatório técnico — versão 0.1 — outubro de 2026

Resumo

Este relatório documenta o primeiro protótipo, em 2D, de um algoritmo para inserir elementos de interface de espessura nula em malhas de elementos finitos representadas por estruturas de dados topológicas compactas (CHE e IBHM). O algoritmo recebe um conjunto de arestas e, para cada uma, cria um quadrilátero degenerado e repara a vizinhança de seus dois vértices aplicando um entre quatro operadores — **Canônico**, **Split Element**, **Expand Edge** e **Open Hole** —, escolhido exclusivamente a partir do *estado do vértice*. Descrevemos o problema, a estrutura IBHM, a arquitetura do algoritmo, cada operador com seus detalhes de implementação e o algoritmo de travessia, tal como implementados na classe `CHEInterfaceElementBuilder`. Cada caso é ilustrado com figuras produzidas pelo próprio código. Em seguida, confrontamos a implementação com a apresentação de projeto e com o artigo da IBHM, apontando imprecisões. Uma verificação experimental com 600 casos aleatórios confirma, em todas as configurações suportadas, a validade topológica, a contagem exata de vértices novos e a independência da ordem de inserção, mas revela duas falhas: vértices de borda com mais de uma aresta selecionada e entradas que contêm arestas de elementos de interface.

Sumário

1 Introdução **3**
- 1.1 Elementos de interface . . . 3
- 1.2 O problema da inserção . . . 3
- 1.3 Proposta e contribuições deste relatório . . . 4

2 Estruturas de dados: da CHE à IBHM **5**
- 2.1 Half-edges . . . 5
- 2.2 CHE: half-edges implícitas . . . 5
- 2.3 IBHM: malhas híbridas . . . 6
- 2.4 O que está implementado no repositório . . . 6

3 Arquitetura do algoritmo **7**
- 3.1 Princípios de projeto . . . 7
- 3.2 Componentes . . . 8
- 3.3 O gabarito do elemento de interface . . . 8
- 3.4 Estados de um vértice e escolha do operador . . . 9

*Texto redigido com o auxílio do Claude Code a partir do código do ramo `feature/edge-element-insertion` (commit `ce3461d`), da apresentação `doc/ppt/Operators_Template` e do artigo da IBHM [1]. As figuras de malha foram geradas executando o próprio código (Apêndice A).

4 Os operadores **10**
4.1 Canônico . . . 11
4.2 Split Element . . . 11
4.3 Expand Edge . . . 12
4.4 Open Hole . . . 13
4.5 Relações entre os operadores . . . 14
4.6 Contagem prévia de vértices . . . 15

5 O algoritmo de travessia **15**
5.1 Inserção de um lote de arestas . . . 15
5.2 O exemplo completo, passo a passo . . . 16
5.3 Propriedades observadas . . . 16
5.4 A variante para a IBHM . . . 18

6 Verificação experimental **18**
6.1 Resultados . . . 19
6.2 O caso não suportado: vértice de borda com duas arestas . . . 19

7 Documentação × implementação **20**
7.1 Apresentação de projeto (`Operators_Template`) . . . 20
7.2 Comentários do código . . . 21
7.3 Artigo da IBHM . . . 22
7.4 Arquivos de dados . . . 22
7.5 Defeitos e limitações encontrados no código . . . 22

8 Considerações finais e próximos passos **23**
8.1 Um invariante que resume o algoritmo . . . 23
8.2 Rumo ao 3D . . . 24
8.3 Próximos passos sugeridos . . . 24

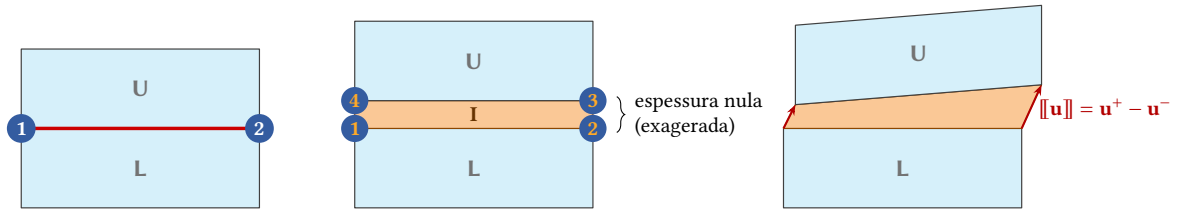
A Reprodução das figuras e dos testes **25**

1 Introdução

1.1 Elementos de interface

Em muitos problemas de engenharia o comportamento que mais importa acontece em superfícies de descontinuidade: fraturas e falhas geológicas, juntas em maciços rochosos, contatos entre materiais distintos, caminhos de propagação de trincas. No método dos elementos finitos, uma forma clássica de representar essas descontinuidades é inserir, ao longo delas, *elementos de interface* — também chamados de elementos de espessura nula ou, quando associados a uma lei de tração–separação, elementos coesivos [6].

Em 2D, um elemento de interface linear é um quadrilátero de quatro nós cujos nós vêm aos pares geometricamente coincidentes (Figura 1). Dois nós pertencem a um lado da descontinuidade e dois ao outro. Como a espessura é nula, a grandeza cinemática relevante deixa de ser a deformação e passa a ser o *salto de deslocamento* entre os dois lados, $[\![\mathbf{u}]\!] = \mathbf{u}^+ - \mathbf{u}^-$, que mede abertura e deslizamento da interface.



(a) malha contínua (b) elemento de interface (c) abertura da interface

Figura 1: O elemento de interface. (a) Dois elementos contínuos compartilham uma aresta (em vermelho). (b) A aresta é *aberta*: os nós 1 e 2 são duplicados em 4 e 3, e entre U e L surge um quadrilátero de quatro nós com $\mathbf{x}_1 = \mathbf{x}_4$ e $\mathbf{x}_2 = \mathbf{x}_3$ (desenhado com espessura exagerada). (c) Durante a simulação os dois lados podem se separar; o elemento mede o salto de deslocamento.

Há duas estratégias para colocar esses elementos na malha. Na estratégia *a priori*, todos os elementos de interface são criados antes da análise, ao longo de todas as superfícies em que uma descontinuidade pode ocorrer. Na estratégia *adaptativa*, eles são inseridos durante a simulação, apenas onde e quando um critério (de tensão, de dano etc.) indica que a descontinuidade se formou [6]. A segunda estratégia evita a rigidez artificial e o custo de milhares de elementos inúteis, mas exige um algoritmo que altere a topologia da malha de forma incremental, local e eficiente, sobre uma malha que *já contém* elementos de interface.

1.2 O problema da inserção

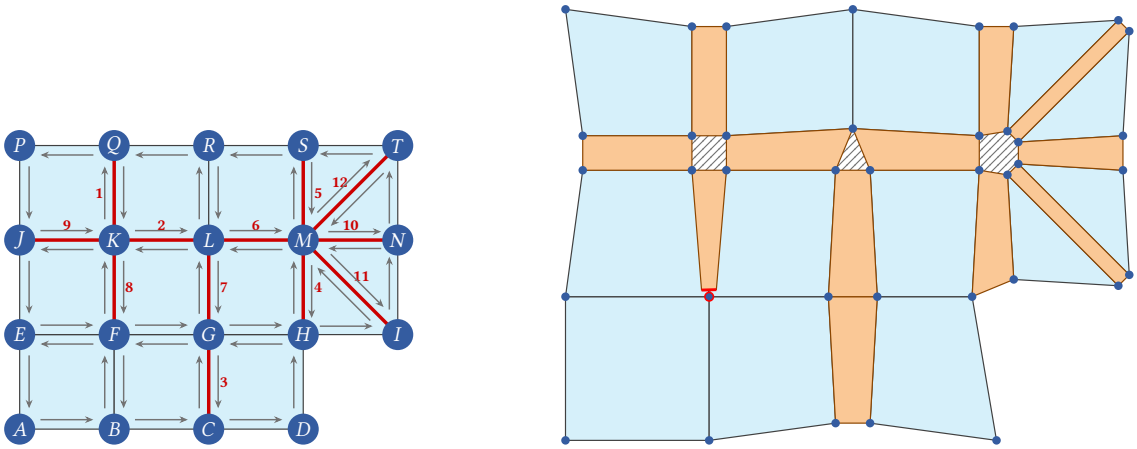
Problema

Dados: uma malha 2D orientada $\mathscr{M}$ e um conjunto S de arestas internas (cada uma compartilhada por dois elementos).
Obter: uma malha $\mathscr{M}'$ em que cada aresta de S foi *aberta*: os dois elementos que a compartilhavam passam a se ligar através de um novo elemento de interface de quatro nós, e os vértices são duplicados apenas o necessário para que a topologia continue consistente.

A Figura 2 mostra a entrada e a saída para o exemplo usado ao longo de todo o texto: uma malha híbrida de quadriláteros e triângulos, com 12 arestas selecionadas e numeradas na ordem de inserção. Parece simples, mas há várias dificuldades:

1. **Quantas cópias de cada vértice?** Um vértice interno tocado por uma única aresta selecionada não precisa ser duplicado geometricamente (a trinca termina nele). Com duas arestas, ele vira dois. Com três ou mais, surge uma *junção* (em T, em X, em estrela), e os elementos de interface não conseguem fechar a vizinhança sozinhos: aparece um *buraco* poligonal, visível como as regiões hachuradas na Figura 2b.

2. **Quais elementos vão para cada cópia?** A estrela do vértice precisa ser dividida em setores delimitados pelas arestas selecionadas, e cada setor deve passar a usar uma cópia diferente.
3. **Borda.** Na borda da malha a abertura não precisa ser fechada: o vértice sempre ganha uma cópia a mais.
4. **Incrementalidade.** Em uma análise adaptativa as arestas chegam aos poucos e em qualquer ordem. O resultado não deveria depender dessa ordem.
5. **Estruturas compactas.** Estruturas baseadas em índices, como a CHE e a IBHM [2, 1], armazenam a malha em vetores de inteiros, sem ponteiros. Os próprios autores da IBHM apontam como limitação o uso em remalhamento dinâmico [1]. Inserir elementos é exatamente esse tipo de alteração.
6. **Extensão para 3D.** O objetivo final é inserir elementos em faces de malhas 3D. As decisões tomadas em 2D devem ser transportáveis.



(a) Entrada: malha com as arestas selecionadas (em vermelho) e a ordem de inserção.

(b) Saída produzida pelo código: 12 elementos de interface (laranja), 19 vértices novos e três buracos (hachurados).

Figura 2: O exemplo da apresentação de projeto [7], executado pelo código. Na saída, cada cópia de um vértice foi deslocada um pouco na direção dos elementos que a usam, para que vértices coincidentes fiquem visíveis ("vista explodida"). A malha é a do arquivo `malha2Hyb.txt`.

1.3 Proposta e contribuições deste relatório

O algoritmo estudado aqui [7] parte de uma observação simples, que dá nome a um dos slides de projeto: embora o problema seja *abrir arestas e criar elementos*, as decisões difíceis acontecem nos *vértices*. Por isso, a inserção de cada aresta é decomposta em duas aplicações de operadores locais, uma em cada extremidade, e o operador é escolhido olhando apenas para o estado daquele vértice. Há quatro operadores (**Canônico**, **Split Element**, **Expand Edge** e **Open Hole**), e a ordem das arestas é livre.

Este relatório tem quatro objetivos:

- explicar o problema e a estrutura de dados IBHM de forma acessível (Seções 1 e 2);
- descrever a arquitetura do algoritmo, cada operador e o algoritmo de travessia exatamente como estão implementados na classe `CHEInterfaceElementBuilder` (Seções 3 a 5);
- verificar experimentalmente o comportamento do código (Seção 6);
- confrontar a documentação existente (apresentação, comentários do código e artigo da IBHM) com a implementação, apontando imprecisões e defeitos (Seção 7).

A Seção 8 reúne próximos passos, inclusive observações para a extensão 3D.

2 Estruturas de dados: da CHE à IBHM

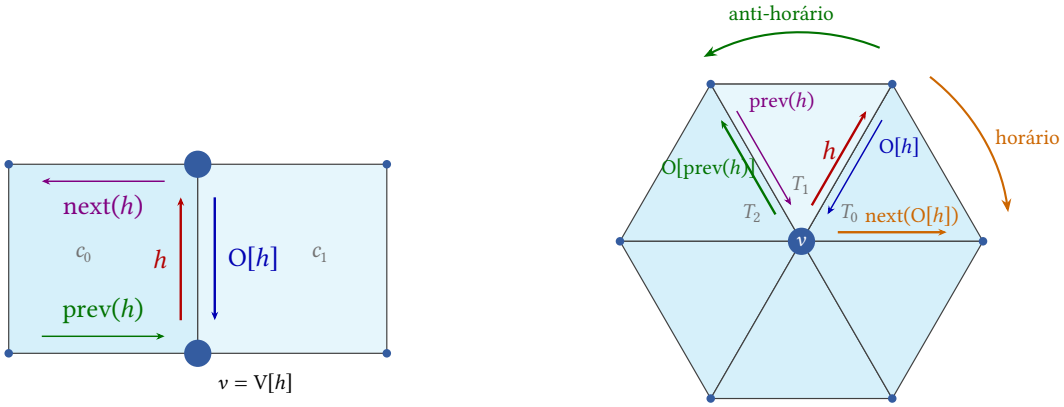
O algoritmo opera inteiramente sobre a topologia da malha. Esta seção apresenta o vocabulário necessário: o conceito de *half-edge*, a estrutura compacta CHE, sua extensão para malhas híbridas (IBHM) e como as duas estão implementadas no repositório.

2.1 Half-edges

Em uma malha 2D orientada, cada elemento é um polígono cujos vértices são listados no sentido anti-horário. Cada lado do elemento, visto com a orientação induzida pelo elemento, é uma *half-edge* [4]. Uma aresta interna é, portanto, representada por duas half-edges com sentidos opostos, uma em cada elemento vizinho; uma aresta de borda tem apenas uma. Cada half-edge h guarda o vértice de onde parte, V[h]. A partir dela definem-se três consultas (Figura 3a):

- next(h) e prev(h): a half-edge seguinte e a anterior no mesmo elemento;
- O[h]: a half-edge *oposta*, isto é, a mesma aresta vista pelo elemento vizinho (ou BORDER se h está na borda).

Note que h parte de V[h] e chega em V[next(h)], e que O[h] vai de V[next(h)] para V[h].



(a) Consultas básicas. **(b)** Girando em torno de um vértice v.

Figura 3: (a) Uma half-edge h do elemento c_0 e suas consultas. (b) As duas maneiras de girar em torno de um vértice usando apenas next, prev e O: a regra $h \leftarrow$ O[prev(h)] passa ao elemento vizinho no sentido anti-horário (usada por `reindexElementsCCW`), e a regra $h \leftarrow$ next(O[h]) passa ao vizinho no sentido horário (usada por `isBorder` e pelo segundo laço de `openHole`). Em ambos os casos a nova half-edge também parte de v.

2.2 CHE: half-edges implícitas

A *Compact Half-Edge* (CHE) [2] dispensa objetos "half-edge". Se todos os elementos têm k vértices, basta guardar a lista de elementos como um vetor de inteiros V[ ] com k entradas por elemento: a posição h desse vetor *é* a half-edge, e o valor armazenado é o vértice de origem. As demais consultas saem por aritmética:

$$\text{elem}(h) = \lfloor h/k \rfloor, \qquad \text{next}(h) = k\,\text{elem}(h) + (h+1) \bmod k, \qquad \text{prev}(h) = k\,\text{elem}(h) + (h+k-1) \bmod k.$$

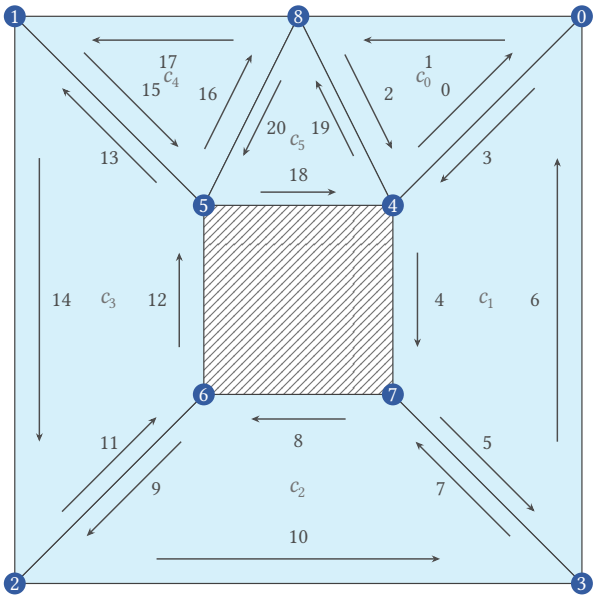
O único vetor adicional é o das opostas, O[ ], com uma entrada por half-edge. Toda a adjacência da malha cabe em dois vetores de inteiros do mesmo tamanho.

2.3 IBHM: malhas híbridas

A IBHM (*Index-Based data structure for Hybrid Meshes*) [1] estende a CHE (e a CHF [3], em 3D) para malhas com elementos de tipos diferentes. Em 2D, a mudança essencial é um vetor de *offsets* $Of[\,]$ que diz onde cada elemento começa em V[ ]. Assim, o elemento c ocupa as posições $Of[c], \ldots, Of[c] + n_v(c) - 1$, e:

- elem(h) é obtido por *busca binária* em $Of[\,]$, em $O(\log n_c)$ (em vez de uma divisão);
- next(h) $= Of[c] + \big((h - Of[c] + 1) \bmod n_v(c)\big)$ e analogamente para prev;
- O[ ] é idêntico ao da CHE;
- um vetor extra $VH[\,]$ associa a cada vértice uma half-edge que parte dele (a de borda, se o vértice estiver na borda), o que permite iniciar consultas de vizinhança e testar borda em $O(1)$.

A Figura 4 mostra os vetores para a malha do arquivo `meshIBHM.txt`, que reproduz (com índices a partir de zero) o exemplo da Figura 4 do artigo: um quadrado com um furo quadrado, formado por três quadriláteros e três triângulos.



he	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
$V[\,]$	4	0	8	0	4	7	3	3	7	6	2	2	6	5	1	1	5	8	5	4	8
$O[\,]$	3	–	19	0	–	7	–	5	–	11	–	9	–	15	–	13	20	–	–	2	16

$$Of[\,] = [0,\ 3,\ 7,\ 11,\ 15,\ 18,\ 21]$$

V[h]: vértice de origem de h; O[h]: oposta de h ("–" = BORDER); $Of[c]$: primeira half-edge do elemento c.

Figura 4: A malha de `meshIBHM.txt` com o índice de cada half-edge e os vetores da IBHM tal como construídos pela classe `IBHM`. O último elemento (c_5) começa num vértice diferente do usado no artigo, por isso suas três half-edges têm outra numeração; o restante coincide com a Figura 4 de [1] subtraindo-se 1 dos índices. O furo central é delimitado por half-edges de borda.

2.4 O que está implementado no repositório

As classes `CHE` e `IBHM` (pasta `core/TopologyDataStructures`) seguem as ideias acima, com algumas escolhas próprias que importam para o algoritmo de inserção (Tabela 1):

Dois pontos merecem destaque. Primeiro, o estado COLLAPSED é uma extensão criada para o algoritmo de inserção: ele permite que um elemento de interface tenha um lado de comprimento zero sem vizinho, e as consultas que giram em torno de vértices (`isBorder`) sabem pular esse lado. Segundo, a combinação "contar → reservar → validar" é a resposta da implementação à limitação apontada no artigo para

Aspecto	Artigo da IBHM [1]	Implementação (CHE/IBHM)
Índices	a partir de 1; O[h] = 0 na borda	a partir de 0; O[h] = BORDER = -1 (como `unsigned`: `0xFFFFFFFF`)
Estado extra de O	—	COLLAPSED = -2 marca uma aresta *colapsada* (de comprimento zero) de um elemento de interface
Offsets	n_c entradas	$n_c + 1$ entradas (sentinela final), de modo que $n_v(c) = Of[c+1] - Of[c]$
elem(h)	busca binária	`std::upper_bound` (IBHM) ou divisão (CHE)
Construção de O	mapa indexado pelo par ordenado de vértices	para cada vértice, lista das half-edges que partem dele; procura-se a half-edge reversa (`buildOppositesLinear`)
Vetor $VH[\,]$	sim	**não**; o teste de borda (`isBorder`) gira em torno do vértice
Crescimento da malha	não previsto ("limitação" declarada)	`reserveSpaceForElements/Nodes` reservam espaço, `addPoint` e `commitElement` validam novos itens; contadores de itens válidos separam o reservado do usado

Tabela 1: Diferenças entre a IBHM do artigo e as classes do repositório.

alterações dinâmicas: o algoritmo de inserção calcula antes quantos vértices e elementos novos serão necessários (Seção 4.6), reserva o espaço de uma vez e só então altera a malha, sem realocações durante as operações.

A interface usada pelo construtor de elementos de interface é pequena: `heVertexIndex` (V), `heOpposite` (O), `heNext`, `hePrevious`, `heElement`, `isBorder`, `setElementVertex` (altera V[h]), `setOpposite` (altera O nos dois sentidos quando a oposta é uma half-edge válida, ou em um só sentido quando é BORDER ou COLLAPSED), `addPoint`, `nextAvailableHE` e `commitElement`. Nenhuma coordenada é consultada: o algoritmo é puramente topológico.

3 Arquitetura do algoritmo

3.1 Princípios de projeto

A apresentação de projeto [7] lista como objetivos "um conjunto simples e pequeno de operadores", resolver a implementação "no nível dos operadores" e um algoritmo que percorra as arestas de entrada aplicando-os. A implementação materializa esses objetivos nas seguintes características:

P1. Baseado em operadores. Inserir uma aresta AB é sempre: criar o elemento, aplicar *um* operador em A, aplicar *um* operador em B e ligar as duas arestas "do meio" (Seção 3.3). Há apenas quatro operadores.

P2. Decisão centrada no vértice. O operador aplicado em um vértice depende só do estado desse vértice (pertence a algum elemento de interface? tem uma aresta colapsada? essa aresta compartilha um elemento com a nova aresta?). Não há análise de combinações de arestas nem dependência da ordem de chegada.

P3. Localidade. Cada operador altera somente a estrela de um vértice; o custo é $O(1)$ ou proporcional à valência.

P4. Puramente topológico. Nenhuma coordenada, ângulo ou produto vetorial é usado: só V, O, next e prev. A apresentação justifica explicitamente essa escolha pela futura extensão a 3D.

P5. Duplicação adiada. O primeiro contato com um vértice interno *não* cria vértice novo: o elemento de interface ganha uma aresta *colapsada* (de comprimento zero) naquele vértice. A cópia só é criada quando, e se, uma segunda aresta chegar. É isso que faz uma ponta de trinca ficar fechada e que torna a contagem de vértices exata.

P6. Incremental. O construtor guarda o estado necessário entre chamadas; novas arestas podem ser inseridas a qualquer momento sobre uma malha que já tem elementos de interface (inserção adaptativa).

P7. Memória previsível. O número de vértices novos é calculado antes de alterar a malha, e todo o espaço é reservado de uma vez (Seção 4.6).

3.2 Componentes

A classe `CHEInterfaceElementBuilder` recebe um ponteiro para a malha e mantém três estruturas auxiliares, que formam uma "segunda camada" de informação sobre a malha:

Estado	Significado
`_inInterfaceElement[v]`	o vértice v pertence a algum elemento de interface (por vértice).
`_isInterfaceElement[e]`	o elemento e é um elemento de interface (por elemento).
`_collapsed`	mapa $v \mapsto c$: vértices que têm uma aresta colapsada e a half-edge colapsada c (que vai de v para v).

Tabela 2: Estado mantido pelo construtor entre inserções.

Como esse estado não está guardado na malha, deve existir um único construtor por malha durante toda a sessão de edição (os comentários da variante IBHM registram essa restrição). Os métodos se dividem em quatro grupos:

- **travessia**: `insertInterfaceElements` percorre as arestas de entrada (Seção 5);
- **decisão**: `retrieveOperator` escolhe o operador, com o auxílio de `retrieveAvailableVertex` para distinguir **Split Element** de **Expand Edge**;
- **operadores**: `canonical`, `splitElement`, `expandEdge` e `openHole`;
- **apoio**: `reindexElementsCCW` (troca o vértice de um leque de elementos) e `computeNumberOfNewVertices` (pré-contagem).

3.3 O gabarito do elemento de interface

Todo elemento de interface é criado com o mesmo gabarito (Figura 5). Seja a a half-edge selecionada, que vai de A para B no elemento U, e $b = \mathrm{O}[a]$, que vai de B para A no elemento L. O novo elemento recebe as quatro próximas half-edges livres, $h_0, \ldots, h_3$, cujos vértices de origem são F, G, H, E:

- $h_0 : F \to G$ e $h_2 : H \to E$ são as arestas "do meio", paralelas a AB. Ao final, h_0 vira oposta de b e h_2 vira oposta de a; a e b deixam de ser opostas entre si.
- $h_3 : E \to F$ é o lado curto em A; E e F são cópias (topológicas) de A, E do lado de U e F do lado de L.
- $h_1 : G \to H$ é o lado curto em B; G é a cópia de B do lado de L, e H, do lado de U.

O operador do vértice A é chamado com o par (a, h_3) e o do vértice B com (b, h_1). Em ambos os casos vale a mesma convenção: *a origem de `elementHE` fica do lado do elemento que contém `edgeHE`, e* next(`elementHE`) *fica do outro lado.* Cada operador, portanto, só precisa decidir dois vértices, V[`elementHE`] e V[next(`elementHE`)], reindexar o que for preciso na estrela do vértice e definir O[`elementHE`]. As arestas

do meio são sempre tratadas da mesma forma pela travessia, qualquer que seja o operador (slide “What happens in the middle?”).

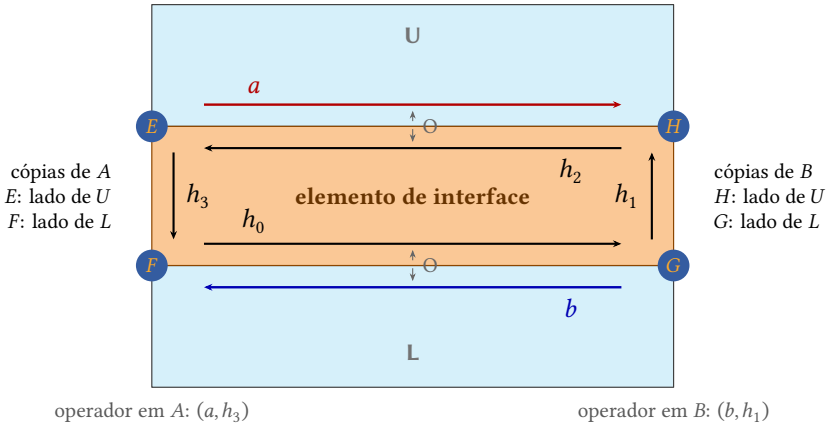


Figura 5: Gabarito de todo elemento de interface ($FGHE$, sentido anti-horário), desenhado com espessura exagerada. As arestas do meio h_0 e h_2 são ligadas a b e a pela travessia; os lados curtos h_3 (em A) e h_1 (em B) são responsabilidade dos operadores. Quando A e B são internos e ainda não pertencem a elemento de interface, o elemento resultante é $[A, B, B, A]$, com h_1 e h_3 colapsadas.

3.4 Estados de um vértice e escolha do operador

Do ponto de vista de um vértice original, a sequência de inserções o faz passar por uma pequena máquina de estados (Figura 6). Um vértice interno começa *regular*; a primeira aresta o deixa *colapsado* (um elemento de interface, nenhuma cópia nova); a segunda o *divide* em duas cópias, que ficam unidas pelos lados curtos dos dois elementos de interface; a terceira e as seguintes abrem e aumentam um *buraco*. Um vértice de borda é dividido já na primeira aresta, porque a abertura escapa para o exterior. O caso de uma segunda aresta em um vértice de borda não é tratado pela implementação atual (Seção 6.2).

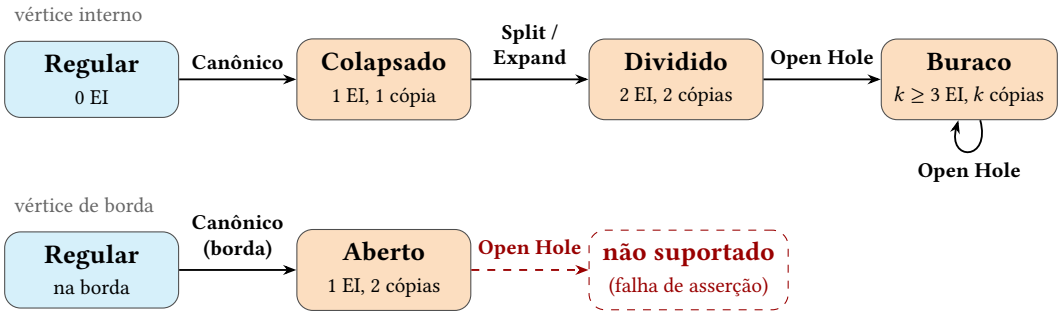


Figura 6: Estados de um vértice original ao longo das inserções (EI = elemento de interface). Os rótulos das transições são os operadores aplicados. Com k arestas selecionadas, um vértice interno termina com $\max(1, k)$ cópias e um de borda com $k + 1$ (este último só é atingido hoje para $k = 1$).

A implementação não guarda esse estado explicitamente; ele é deduzido em `retrieveOperator` a partir das três estruturas da Tabela 2, olhando para o vértice v de onde parte a half-edge recebida (Figura 7):

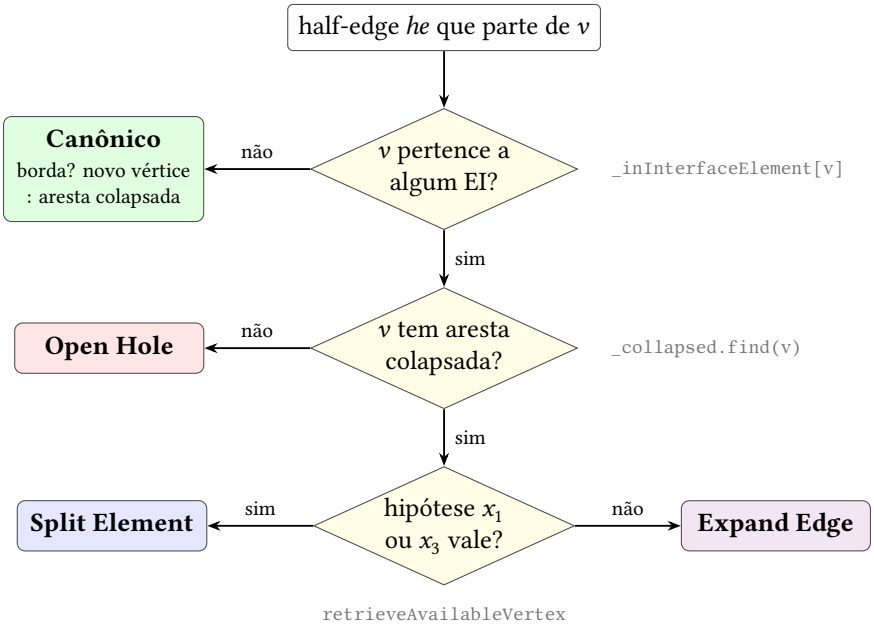


Figura 7: Árvore de decisão de `retrieveOperator`. As hipóteses x_1 e x_3 verificam se o elemento de interface que tem a aresta colapsada em v é vizinho de um dos dois elementos que contêm a aresta nova (Seção 4.2).

Operador	Condição em $v = \mathrm{V}[he]$	novos	V[eHE], V[next(eHE)]	O[eHE]	custo
Canônico (interno)	v fora de EI, interno	0	v, v	COLLAPSED	$O(\deg v)^*$
Canônico (borda)	v fora de EI, na borda	1 (v')	v', v	BORDER	$O(\deg v)$
Split Element (x_1)	colapsado, x_1 vale	1 (n)	n, v	c	$O(1)$
Split Element (x_3)	colapsado, x_3 vale	1 (n)	v, n	c	$O(1)$
Expand Edge	colapsado, nenhuma hipótese	1 (n)	v, n	c	$O(\deg v)$
Open Hole	em EI, sem colapsada	1 (n)	n, v	BORDER	$O(\deg v)$

Tabela 3: Resumo dos operadores. "eHE" é a half-edge `elementHE` do novo elemento (h_3 em A, h_1 em B); c é a half-edge colapsada registrada em `_collapsed[v]`, que deixa de ser colapsada e passa a ser a oposta de eHE. *O custo do Canônico interno vem apenas do teste `isBorder`, que gira em torno do vértice (na IBHM, $O(\deg v \cdot \log n_c)$ por causa da busca binária).

4 Os operadores

Esta seção descreve os quatro operadores tal como implementados. Cada um recebe a half-edge da aresta nova que parte do vértice v (chamada `edgeHE` ou `he` no código) e a half-edge do lado curto correspondente do novo elemento (`elementHE`, h_3 ou h_1). Nos pseudocódigos, $\mathrm{V}[h] \leftarrow w$ corresponde a `setElementVertex(h, w)`, "$\mathrm{O}[x] \leftrightarrow y$" a `setOpposite(x, y)` com y uma half-edge válida (liga nos dois sentidos) e `addPoint()` cria um vértice novo (sem coordenadas: hoje ele é colocado na origem).

As figuras desta seção foram geradas executando o código em malhas pequenas (Apêndice A). Convenções visuais: elementos contínuos em azul, elementos de interface em laranja, aresta a inserir em vermelho; vértices com aresta colapsada têm contorno vermelho; vértices que pertencem a algum elemento de interface têm rótulo laranja; as cópias de um vértice X são chamadas $X_1, X_2, \ldots$ na ordem de criação e são desenhadas ligeiramente afastadas umas das outras. Arestas colapsadas (comprimento zero) aparecem como um pequeno segmento vermelho, com uma espessura *apenas visual*.

Todos os operadores usam a rotina de apoio `reindexElementsCCW` (Algoritmo 1), que gira no sentido anti-horário a partir de uma half-edge trocando o vértice de origem até encontrar uma half-edge de parada, a borda ou o ponto de partida.

Algoritmo 1 `reindexElementsCCW`$(h, stop, w)$

```
1: s ← h
2: repita
3:     V[h] ← w                          ▷ o canto de h passa a usar o vértice w
4:     h ← O[prev(h)]                    ▷ próximo elemento no sentido anti-horário
5: até que h = stop ou h = BORDER ou h = s
```

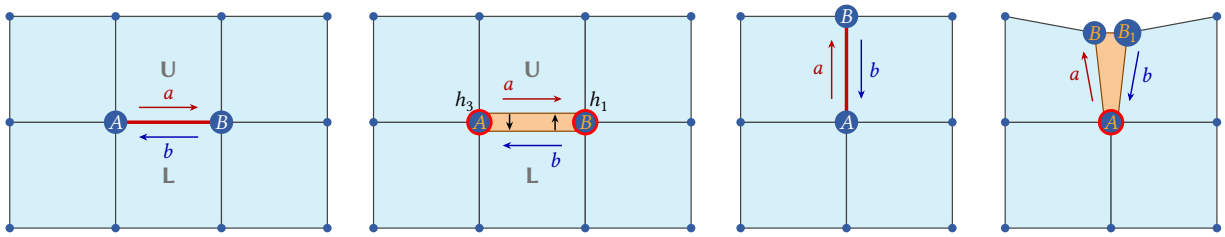
4.1 Canônico

Quando: o vértice v ainda não pertence a nenhum elemento de interface. É a "inicialização" do vértice.

O que faz: se v é interno, nada é duplicado: o lado curto do novo elemento vira uma aresta colapsada de v para v, com oposta COLLAPSED, e ela é registrada em `_collapsed[v]`. A trinca termina em v e o elemento de interface fica "fechado" ali. Se v está na borda, a abertura chega ao exterior: cria-se um vértice v', todos os elementos do lado de `edgeHE` até a borda passam a usá-lo (rotação anti-horária), e o lado curto do novo elemento fica na borda (Algoritmo 2 e Figura 8).

Algoritmo 2 `canonical`(e, x) $\quad$ (e = `edgeHE`, x = `elementHE`)

```
1: v ← V[e]
2: se isBorder(e) então                 ▷ gira em torno de v procurando uma half-edge sem oposta
3:     v' ← addPoint()
4:     V[x] ← v'
5:     reindexElementsCCW(e, BORDER, v')                ▷ lado de e, até a borda
6:     O[x] ← BORDER;   _inInterfaceElement[v'] ← true
7: senão
8:     V[x] ← v;   O[x] ← COLLAPSED;   _collapsed[v] ← x
9: V[next(x)] ← v;   _inInterfaceElement[v] ← true
```



(a) Antes: A e B internos. **(b)** Depois: elemento $[A, B, B, A]$. **(c)** Antes: B na borda. **(d)** Depois.

Figura 8: Operador Canônico. (a–b) Com os dois vértices internos, o elemento de interface tem as duas extremidades colapsadas (h_3 em A e h_1 em B, em vermelho) e nenhum vértice é criado: geometricamente ele é só a aresta AB. (c–d) Com B na borda, B é duplicado em B_1; o operador em B recebe b, e os elementos do lado de b (à direita) passam a usar a cópia; o lado curto em B é uma aresta de borda. Em A (interno) a extremidade continua colapsada.

4.2 Split Element

Quando: v tem uma aresta colapsada c (pertence a exatamente um elemento de interface I, criado pelo Canônico) *e* I é vizinho de um dos dois elementos que contêm a aresta nova. Esse elemento comum é o *elemento compartilhado* S.

Como o código testa (`retrieveAvailableVertex`, Algoritmo 3). Seja he a half-edge nova que parte de v e $b = \mathrm{O}[he]$. Há duas posições possíveis para S:

- **hipótese** x_1: S é o elemento de he. Então a aresta de S que chega em v, prev(he), é vizinha de I, e sua oposta $x_1 = \mathrm{O}[\mathrm{prev}(he)]$ é o lado longo de I que *sai* de v logo depois de c, isto é, $\mathrm{prev}(x_1) = c$;
- **hipótese** x_3: S é o elemento de b. Então a aresta de S que sai de v, next(b), é vizinha de I, e $x_3 = \mathrm{O}[\mathrm{next}(b)]$ é o lado longo de I que *chega* em v logo antes de c, isto é, $\mathrm{next}(x_3) = c$.

Qual hipótese ocorre não depende de qual half-edge da aresta o usuário escolheu, e sim da posição da aresta nova em relação a I: em S, se a aresta nova vem depois da aresta de I (no sentido anti-horário de S) vale x_1; se vem antes, vale x_3 (Figura 9).

O que faz (Algoritmo 3): cria um vértice n e o coloca "dentro" de S: só dois cantos mudam, o canto de S em v e o canto de I adjacente a S. A antiga aresta colapsada c deixa de ter comprimento zero (passa a ligar v e n) e vira a fronteira entre I e o novo elemento, sendo ligada a `elementHE`. O custo é $O(1)$.

Algoritmo 3 `retrieveAvailableVertex`(he) e `splitElement`(he, av, sh, x)

função RETRIEVEAVAILABLEVERTEX(he)
  $v \leftarrow \mathrm{V}[he]$; $b \leftarrow \mathrm{O}[he]$; $c \leftarrow$ `_collapsed`$[v]$
  $x_1 \leftarrow \mathrm{O}[\mathrm{prev}(he)]$
  se $x_1 \neq$ BORDER **e** $\mathrm{prev}(x_1) = c$ **então retorne** $(av = x_1,\ sh = he)$ ▷ hipótese x_1
  $x_3 \leftarrow \mathrm{O}[\mathrm{next}(b)]$
  se $x_3 \neq$ BORDER **e** $\mathrm{next}(x_3) = c$ **então retorne** $(av = c,\ sh = \mathrm{next}(b))$ ▷ hipótese x_3
  retorne (BORDER, BORDER) ▷ nenhum elemento compartilhado ⇒ Expand Edge

procedimento SPLITELEMENT(he, av, sh, x)
  $v \leftarrow \mathrm{V}[he]$; $n \leftarrow$ `addPoint`()
  $\mathrm{V}[av] \leftarrow n$ ▷ canto do elemento de interface antigo
  $\mathrm{V}[sh] \leftarrow n$ ▷ canto do elemento compartilhado
  se $he = sh$ **então** ▷ hipótese x_1
    $\mathrm{V}[x] \leftarrow n$; $\mathrm{V}[\mathrm{next}(x)] \leftarrow v$; $\mathrm{O}[x] \leftrightarrow \mathrm{prev}(av)$ ▷ $\mathrm{prev}(av) = c$
  senão ▷ hipótese x_3
    $\mathrm{V}[x] \leftarrow v$; $\mathrm{V}[\mathrm{next}(x)] \leftarrow n$; $\mathrm{O}[x] \leftrightarrow av$ ▷ $av = c$
  marca v e n como vértices de EI; remove v de `_collapsed`

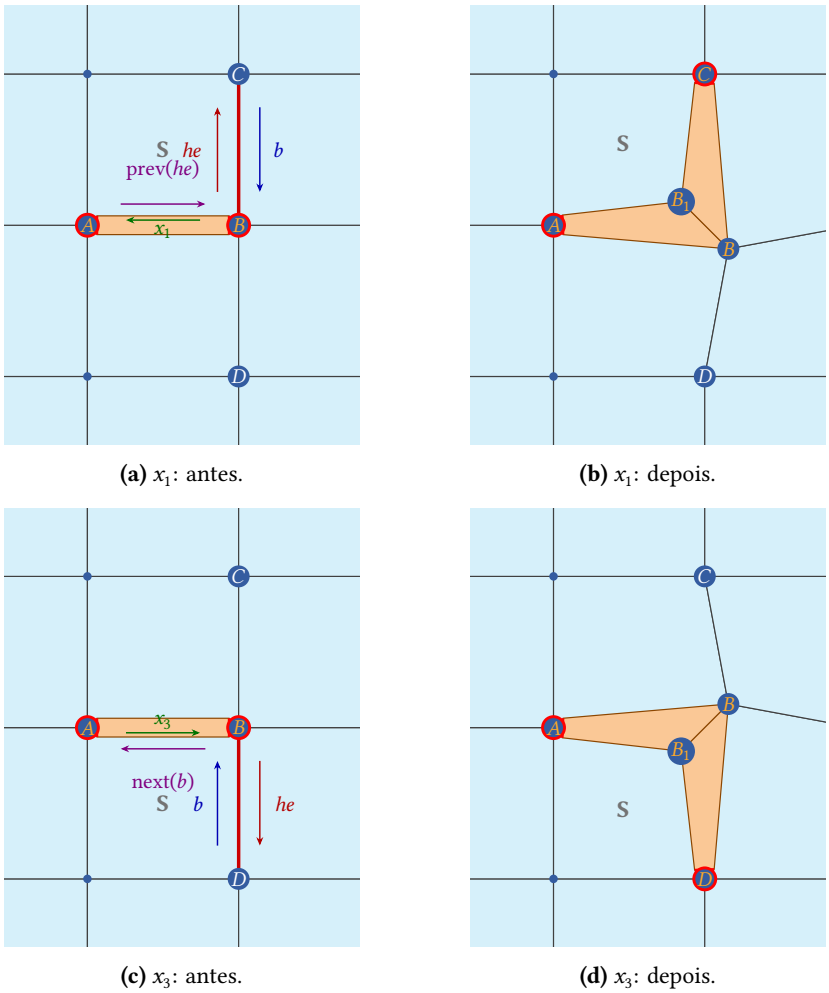
4.3 Expand Edge

Quando: v tem uma aresta colapsada c, mas nenhuma das duas hipóteses vale: o elemento de interface I não toca os elementos da aresta nova. É o caso típico de uma trinca que continua "em frente" através de v (Figura 10).

O que faz (Algoritmo 4): a estrela de v precisa ser dividida em dois setores, delimitados pela aresta nova de um lado e por I do outro. O operador cria n e gira no sentido anti-horário *a partir da aresta colapsada* c: primeiro troca o canto de c em I, depois sai de I pelo seu lado longo $\mathrm{prev}(c)$ e percorre os elementos daquele setor até alcançar he, que *não* é alterado. Assim, a parada natural da rotação é justamente cruzar a aresta nova. A aresta c deixa de ser colapsada e é ligada a `elementHE`.

Algoritmo 4 `expandEdge`(he, x)

$v \leftarrow \mathrm{V}[he]$; $n \leftarrow$ `addPoint`(); $c \leftarrow$ `_collapsed`$[v]$
`reindexElementsCCW`(c, he, n) ▷ de c até he (exclusive)
$\mathrm{O}[c] \leftrightarrow x$
$\mathrm{V}[x] \leftarrow v$; $\mathrm{V}[\mathrm{next}(x)] \leftarrow n$
marca v e n como vértices de EI; remove v de `_collapsed`



(a) x_1: antes. **(b)** x_1: depois.

(c) x_3: antes. **(d)** x_3: depois.

Figura 9: Split Element em B. Nos dois casos já existe um elemento de interface em AB, colapsado em A e em B. (a–b) A aresta nova BC sobe: o elemento compartilhado S (acima de AB) contém he, e o lado de I que sai de B é x_1. (c–d) A aresta nova BD desce: agora S (abaixo de AB) contém b, e o lado de I que chega em B é x_3. Em ambos, só S e um canto de I passam a usar a nova cópia B_1, e a antiga aresta colapsada em B vira o lado comum entre os dois elementos de interface. Note que C e D, internos, ficam colapsados.

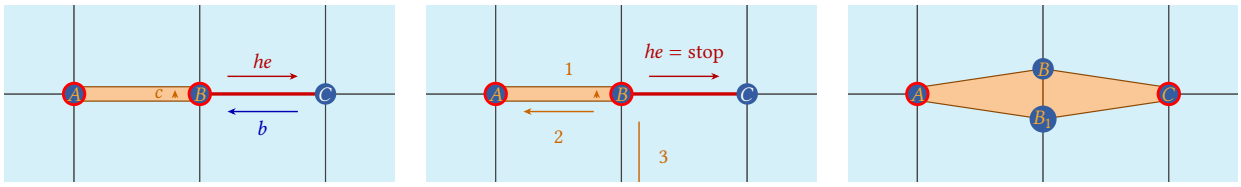
4.4 Open Hole

Quando: v já pertence a um elemento de interface, mas não tem aresta colapsada. Isso acontece a partir da terceira aresta em um vértice interno: depois do Split/Expand o vértice original já virou duas cópias, v e w, e os dois elementos de interface J e K que passam por ele estão colados pelos seus lados curtos (um vai de v para w, o outro de w para v).

O que faz (Algoritmo 5): com três ou mais elementos de interface chegando ao mesmo ponto, os lados curtos não conseguem mais fechar a vizinhança, e aparece um buraco. O operador cria n e faz duas travessias a partir de he:

1. no sentido *anti-horário*, troca v por n em todos os elementos até encontrar o primeiro elemento de interface J; troca também o canto de J e "descola" o lado curto de J que chega a esse canto (O ← BORDER);
2. no sentido *horário* (regra $h \leftarrow \text{next}(\text{O}[h])$), apenas procura o primeiro elemento de interface K e descola o lado curto de K que sai de v.

O lado curto do novo elemento também fica na borda. Na primeira vez, os lados descolados de J e K eram justamente o par que unia as duas cópias, e o resultado é um buraco triangular (v, n, w); nas vezes seguintes o buraco já existe, os lados encontrados já são de borda e o buraco ganha um vértice (Figura 11).



(a) Antes. **(b)** Rotação a partir de c. **(c)** Depois.

Figura 10: Expand Edge em B: a aresta BC continua a trinca AB, e o elemento de interface em AB não toca os elementos de BC. (b) Ordem em que `reindexElementsCCW` visita os cantos: 1 é a aresta colapsada c (em I), 2 e 3 são os elementos abaixo da trinca; a rotação para ao encontrar he (a half-edge de BC que parte de B). (c) Os cantos visitados passam a usar B_1; A e C continuam colapsados.

Algoritmo 5 `openHole`(he, x)

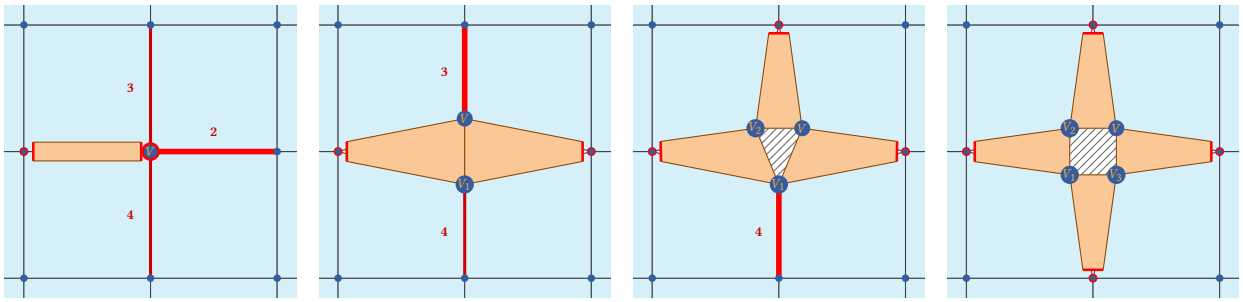
$v \leftarrow \mathrm{V}[he]$; $\quad n \leftarrow$ `addPoint`(); $\quad h \leftarrow he$
repita ▷ laço 1: anti-horário, reindexando
  $\mathrm{V}[h] \leftarrow n$; $\quad h \leftarrow \mathrm{O}[\mathrm{prev}(h)]$
até que $h =$ BORDER **ou** `_isInterfaceElement`$[\mathrm{elem}(h)]$
assert$(h \neq$ BORDER$)$
$\mathrm{V}[h] \leftarrow n$; $\quad \mathrm{O}[\mathrm{prev}(h)] \leftarrow$ BORDER ▷ canto e lado curto de J
$h \leftarrow he$
repita ▷ laço 2: horário, só procura
  $h \leftarrow \mathrm{next}(\mathrm{O}[h])$
até que $h =$ BORDER **ou** `_isInterfaceElement`$[\mathrm{elem}(h)]$
assert$(h \neq$ BORDER$)$
$\mathrm{O}[h] \leftarrow$ BORDER ▷ lado curto de K
$\mathrm{V}[x] \leftarrow n$; $\quad \mathrm{V}[\mathrm{next}(x)] \leftarrow v$; $\quad \mathrm{O}[x] \leftarrow$ BORDER
marca v e n como vértices de EI

4.5 Relações entre os operadores

Olhando os quatro operadores lado a lado, todos fazem a mesma coisa em essência: *dividem um leque da estrela de v entre duas "paredes", dão um vértice novo a um dos lados e decidem o que fazer com os lados curtos.* O que muda é a parede onde a rotação começa e onde ela para:

Operador	começa em	para em	lados curtos
Canônico (borda)	aresta nova	borda da malha	novo lado na borda
Expand Edge	aresta colapsada c	aresta nova	c é colada ao novo lado
Split Element	aresta colapsada c	aresta nova (após 1 elemento)	c é colada ao novo lado
Open Hole	aresta nova	primeiro elemento de interface	descolados (buraco)

Uma consequência interessante: o **Split Element é um caso particular do Expand Edge**. Na hipótese x_3, a rotação do Expand a partir de c visita exatamente o canto de c e o canto $\mathrm{next}(b)$ de S e para em he — as mesmas alterações do Split. Na hipótese x_1, a rotação do Expand vai pelo outro lado e reindexa todos os elementos da estrela *exceto* S; o resultado é a mesma divisão da estrela, só com os nomes v e n trocados. Verificamos isso experimentalmente: substituindo o Split pelo Expand em 665 ocorrências (332 malhas aleatórias), a topologia final foi idêntica em todos os casos (Seção 6). O Split Element é, portanto, uma *otimização*: custo $O(1)$ em vez de $O(\deg v)$, movendo sempre o lado menor. Isso não é um defeito; mas é uma informação útil para a extensão 3D, onde talvez valha começar pela versão geral (rotação) e só depois especializar.



(a) Aresta 1: Canônico. **(b)** Aresta 2: Expand. **(c)** Aresta 3: Open Hole. **(d)** Aresta 4: Open Hole.

Figura 11: Quatro arestas chegando ao vértice interno V (detalhe do centro de uma malha 4×4; a próxima aresta a inserir está em vermelho grosso). (a) A primeira deixa V colapsado. (b) A segunda, alinhada, divide V em duas cópias coladas pelos lados curtos. (c) A terceira descola esses lados: buraco triangular (hachurado). (d) A quarta aumenta o buraco para um quadrilátero. Com k arestas, o buraco tem k vértices.

4.6 Contagem prévia de vértices

Para reservar memória de uma vez, `computeNumberOfNewVertices` conta, para cada vértice v tocado pelo lote de arestas, o número n_v de arestas do lote que o tocam, e aplica:

$$\text{novos}(v) = \begin{cases} n_v - 1, & v \text{ fora de EI e interno (o primeiro contato só colapsa),} \\ n_v, & v \text{ fora de EI e na borda (o primeiro contato já duplica),} \\ n_v, & v \text{ já em algum EI (todo contato duplica).} \end{cases}$$

No código, a regra da borda é implementada somando 1 ao contador *por aresta* que toca um vértice de borda; por isso um vértice de borda com duas arestas no mesmo lote é contado em excesso (Seção 7). Fora esse caso, a contagem foi exata em todos os testes (Seção 6). Os elementos são reservados à razão de um por aresta do lote.

5 O algoritmo de travessia

5.1 Inserção de um lote de arestas

O ponto de entrada é `insertInterfaceElements`, que recebe uma lista de half-edges (uma por aresta; qualquer uma das duas serve) e as processa na ordem dada (Algoritmo 6). Antes de alterar a malha, o método conta os vértices novos e reserva espaço para eles e para um elemento por aresta. Depois, para cada aresta, monta o elemento a partir do gabarito da Figura 5, escolhe e aplica o operador em A e em B e, por fim, liga as arestas do meio.

Algoritmo 6 `insertInterfaceElements`(E) (classe `CHEInterfaceElementBuilder`)

Entrada: lista E de half-edges, uma por aresta a abrir

1: $n_v \leftarrow$ `computeNumberOfNewVertices`(E)
2: reserva espaço para $|E|$ elementos e n_v vértices; aumenta `_isInterfaceElement` e `_inInterfaceElement`
3: **para cada** $a \in E$ **faça**
4: $b \leftarrow \mathrm{O}[a]$
5: **se** $b =$ BORDER **ou** $b =$ COLLAPSED **então continue** ▷ aresta de borda: inválida
6: $h_0 \leftarrow$ `nextAvailableHE`(); $h_1, h_2, h_3 \leftarrow h_0 + 1, h_0 + 2, h_0 + 3$
7: $op \leftarrow$ `retrieveOperator`(a); aplica op com (a, h_3) ▷ vértice A
8: $op \leftarrow$ `retrieveOperator`(b); aplica op com (b, h_1) ▷ vértice B
9: $\mathrm{O}[h_0] \leftrightarrow b$; $\mathrm{O}[h_2] \leftrightarrow a$ ▷ arestas do meio
10: $e \leftarrow$ `commitElement`(); `_isInterfaceElement`$[e] \leftarrow$ **true**

Três detalhes de implementação merecem atenção:

- O operador de B é escolhido *depois* de o operador de A ter sido aplicado. Como os operadores só mexem na estrela do próprio vértice, a ordem A antes de B não altera a decisão em B.
- Enquanto os operadores trabalham, a e b ainda são opostas entre si; é isso que permite que as rotações em torno de A e de B atravessem a aresta que está sendo aberta (por exemplo, o laço horário do Open Hole parte de he e passa por O[he]). Só no final a e b são religadas a h_2 e h_0.
- O elemento só é validado (`commitElement`) depois dos operadores, mas suas half-edges já existem no espaço reservado. Na CHE isso basta, porque next e prev são aritméticos. Na variante IBHM o elemento precisa ser validado *antes*, porque next/prev dependem do vetor de offsets (Seção 5.4).

5.2 O exemplo completo, passo a passo

A Tabela 4 lista o operador escolhido pelo código em cada extremidade, para as 12 arestas do exemplo da apresentação, na ordem dos slides. Os 24 operadores coincidem exatamente com os anunciados nos slides 15 a 26 da apresentação [7]. A Figura 12 mostra o estado da malha depois de cada inserção, e a Figura 13 o resultado final com todos os vértices nomeados.

Aresta	A–B (no código)	Operador em A	Operador em B	vértices novos
1	K–Q	Canônico	Canônico	1
2	K–L	Split Element	Canônico	1
3	G–C	Canônico	Canônico	1
4	M–H	Canônico	Canônico	1
5	S–M	Canônico	Expand Edge	2
6	L–M	Expand Edge	Open Hole	2
7	L_1–G	Open Hole	Expand Edge	2
8	K–F	Open Hole	Canônico	1
9	J–K	Canônico	Open Hole	2
10	M_1–N	Open Hole	Canônico	2
11	M_1–I	Open Hole	Canônico	2
12	M_3–T	Open Hole	Canônico	2
			total	19

Tabela 4: Operadores escolhidos pelo código no exemplo (IBHM, malha `malha2Hyb.txt`). X_i é a i-ésima cópia criada do vértice X. A coluna A–B mostra a orientação da half-edge usada (por isso a aresta 1, "QK" nos slides, aparece como K–Q).

5.3 Propriedades observadas

Independência da ordem. O resultado final não depende da ordem de inserção nem da half-edge escolhida para cada aresta. A razão é que a topologia final é determinada pelo *conjunto* de arestas: as arestas selecionadas cortam a estrela de cada vértice em setores, e cada setor termina com uma cópia própria (salvo no vértice interno com uma única aresta, que fica colapsado). Os operadores sempre dividem a estrela exatamente ao longo das arestas selecionadas, de modo que qualquer ordem leva à mesma partição. Verificamos isso para três ordens do exemplo (a dos slides, a de `main.cpp` e o lote inteiro em uma única chamada) e para três ordens aleatórias em cada um dos 503 casos aleatórios da Seção 6.

Custo. Cada aresta custa $O(\deg A + \deg B)$ na CHE, dominado pelas rotações (`isBorder` no Canônico, reindexação no Expand e no Open Hole); na IBHM há um fator $\log n_c$ da busca binária. A pré-contagem também chama `isBorder` para cada extremidade. O custo total é linear no número de arestas inseridas para malhas de valência limitada, e não depende do tamanho da malha.

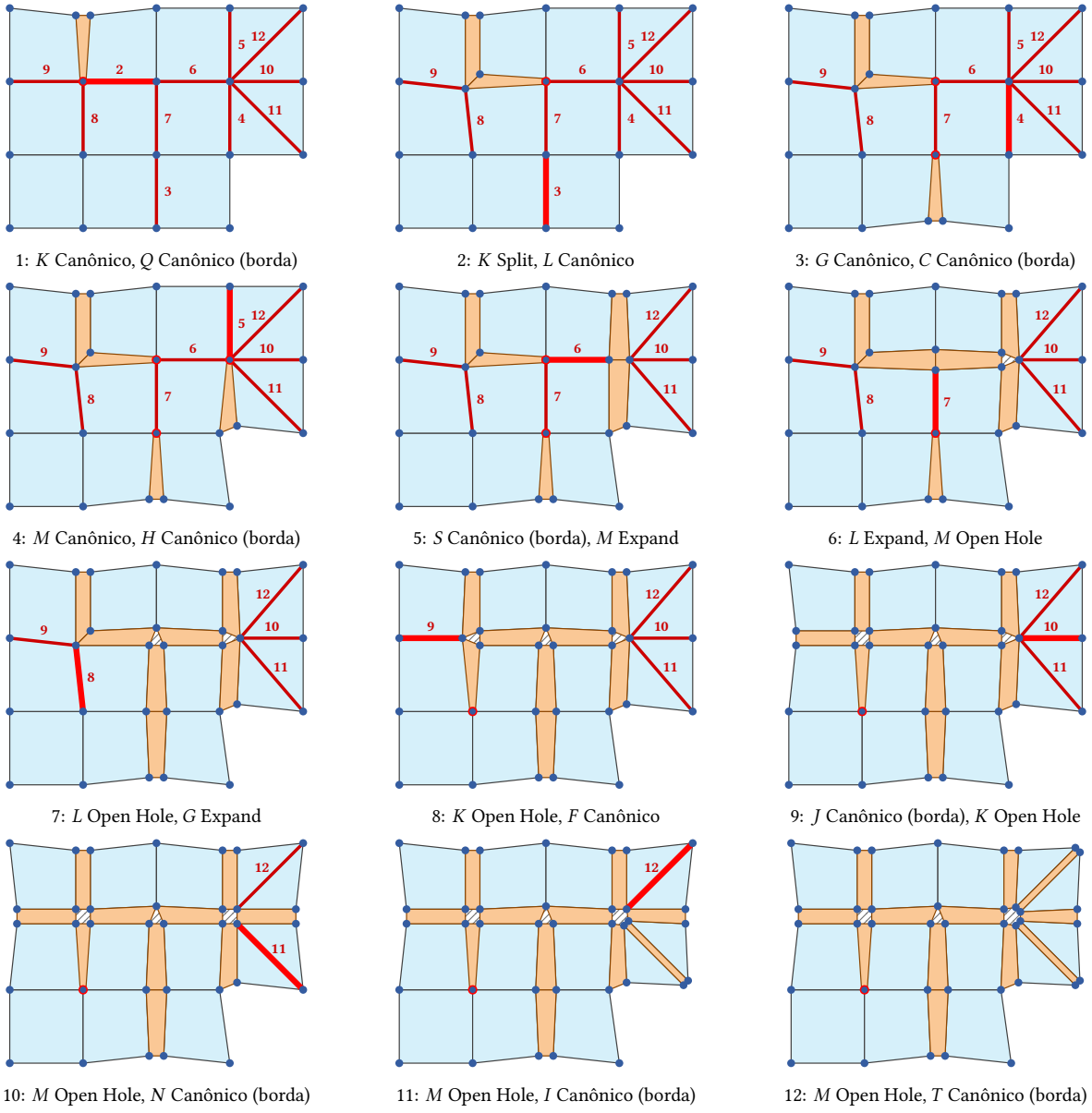


Figura 12: Estado da malha após cada uma das 12 inserções do exemplo (vista explodida). As arestas ainda não inseridas estão em vermelho e a próxima, em vermelho grosso. Observe a sequência em M: colapsado (4), dividido por Expand (5), buraco triangular (6) que cresce até um hexágono (12). Em K a segunda aresta já chega por Split Element (2), porque KL e KQ compartilham o elemento $KLRQ$.

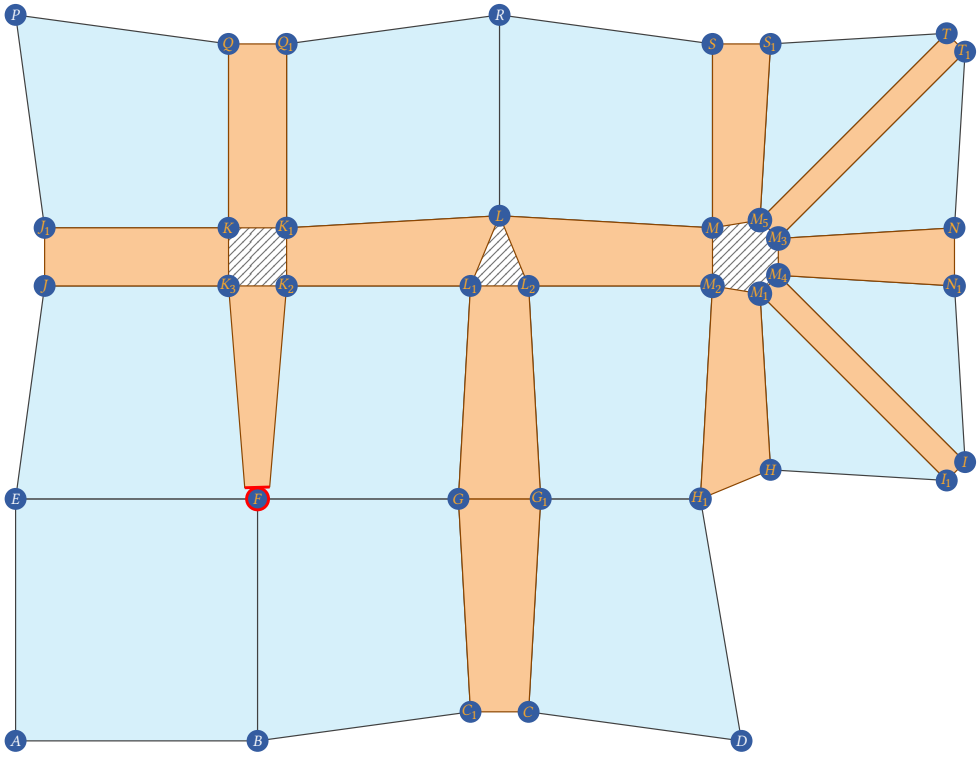


Figura 13: Resultado final do exemplo, com os nomes de todos os vértices. Os vértices K, L e M terminam com 4, 3 e 6 cópias e com buracos de 4, 3 e 6 lados; G (duas arestas) termina com duas cópias coladas; F (uma aresta, interno) continua com uma aresta colapsada (contorno vermelho); os vértices de borda C, H, I, J, N, Q, S e T foram duplicados uma vez. A posição das cópias é apenas ilustrativa: no código todas as cópias novas recebem a coordenada $(0, 0)$.

Memória. O número de vértices novos é exatamente o previsto (exceto pelo caso de borda que não é suportado), e é reservado um elemento por aresta do lote.

5.4 A variante para a IBHM

A classe `IBHMInterfaceElementBuilder` é uma cópia quase literal de `CHEInterfaceElementBuilder` (trocando os nomes das classes, o `diff` entre as duas mostra cerca de 20 linhas diferentes). As diferenças são:

1. o elemento é validado com `commitElement(4)` *antes* dos operadores, porque na IBHM next e prev de uma half-edge só funcionam se o offset do seu elemento já estiver definido;
2. arestas cujos elementos já são de interface são rejeitadas com uma mensagem (a CHE não faz essa verificação);
3. a reserva de espaço é feita em posições de half-edges (4× o número de arestas) e em entradas do vetor de offsets.

Hoje o `main.cpp` executa apenas esta variante, sobre `malha2Hyb.txt`, com as mesmas 12 arestas dos slides em outra ordem (9, 1, 2, 8, 6, 7, 3, 10, 4, 5, 11, 12, na numeração dos slides); o resultado é o mesmo da Figura 13.

6 Verificação experimental

Para confirmar o entendimento descrito nas seções anteriores, escrevemos um pequeno programa de teste (`tools/driver.cpp`, Apêndice A) que compila os arquivos originais do repositório *sem alterações*, insere

arestas uma a uma, registra o operador escolhido em cada extremidade e, após cada inserção, verifica:

V1. simetria das opostas: $O[O[h]] = h$ para toda oposta válida;

V2. consistência de vértices entre opostas: $V[h] = V[\text{next}(O[h])]$ e $V[\text{next}(h)] = V[O[h]]$;

V3. toda half-edge COLLAPSED tem comprimento zero: $V[h] = V[\text{next}(h)]$;

V4. variedade: os cantos de cada vértice formam um único leque conexo pela relação $h \sim O[\text{prev}(h)]$ (nenhum vértice "estrangulado" por dois leques separados);

V5. todo vértice criado é usado por algum elemento;

V6. o número de vértices criados é igual ao previsto por `computeNumberOfNewVertices`.

Para comparar resultados de ordens diferentes, usamos a partição dos cantos dos elementos *originais* em classes de vértices: duas execuções produzem a mesma topologia (a menos de renumeração) se e somente se essas partições coincidem.

6.1 Resultados

Cenário	Resultado
Exemplo dos slides (IBHM, ordem dos slides)	válido em todos os passos; 24/24 operadores iguais aos slides; 19 vértices novos (previsto: 19); buracos em K, L e M.
Exemplo, ordem do `main.cpp` e lote único	mesma topologia da ordem dos slides.
CHE, `malha2.txt`, 10 arestas do `main.cpp`	válido em todos os passos.
Trinca reta J–K–L–M–N em 3 ordens	válida; mesma topologia nas 3 ordens.
Laço fechado em torno de um elemento	válido; Split Element nos quatro cantos.
600 casos aleatórios[a]	503 completos: **todos** válidos, contagem exata, mesma topologia nas 3 ordens. 97 abortados: **exatamente** os 97 casos em que algum vértice de borda recebe duas ou mais arestas.
Split Element substituído por Expand Edge[b]	665 ocorrências em 332 casos: topologia idêntica em todos.
Mesma aresta inserida duas vezes	CHE: insere um segundo elemento de interface colado ao primeiro (Expand nas duas pontas) — válido, mas indesejado. IBHM: rejeita a aresta.
Half-edge de um elemento de interface na entrada	CHE e IBHM: **laço infinito** em `isBorder`, dentro de `computeNumberOfNewVertices`.
Vértice de borda com segunda aresta	falha de asserção em `openHole` (Seção 6.2).

Tabela 5: Resumo da verificação. [a]Grades de 3 a 6 por 3 a 6 elementos; metade só com quadriláteros (CHE) e metade híbrida (IBHM, com cerca de 40% dos quadriláteros divididos em triângulos); um subconjunto aleatório de até metade das arestas internas; três ordens aleatórias por caso e escolha aleatória da half-edge de cada aresta. [b]Variante compilada à parte, em que `retrieveOperator` nunca escolhe Split Element.

Em resumo: *para todas as configurações suportadas, a implementação produz topologias válidas, independentes da ordem, com contagem exata de vértices.* As falhas encontradas estão concentradas em entradas que o projeto ainda não trata, discutidas a seguir e na Seção 7.5.

6.2 O caso não suportado: vértice de borda com duas arestas

Quando um vértice de borda já foi aberto pelo Canônico e recebe uma segunda aresta, ele pertence a um elemento de interface e não tem aresta colapsada; `retrieveOperator` escolhe então Open Hole. Mas um dos dois laços de `openHole` chega à borda da malha antes de encontrar um elemento de interface (Figura 14). Em modo *debug*, a asserção `currentHE != CHE::BORDER` interrompe o programa (os comentários do código

perguntam "*@todo Is it possible?*": sim, é possível). Em modo *release* o resultado é pior: o primeiro laço escreve na posição `0xFFFFFFFF` do vetor de vértices, e o segundo laço nunca detecta a borda, porque testa next(O[h]) em vez de O[h], e next(BORDER) $\neq$ BORDER.

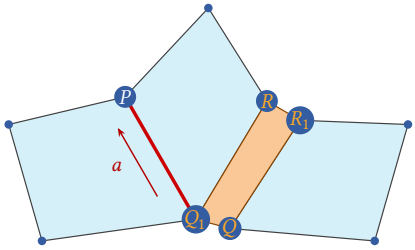


Figura 14: Configuração que falha. Q está na borda e já foi aberto pela aresta QR (cópias Q e Q_1; os dois elementos à esquerda de QR passaram a usar Q_1). Ao inserir QP, o Open Hole em Q_1 gira no sentido anti-horário a partir de *he* e alcança imediatamente a borda (à esquerda), sem encontrar elemento de interface. Nesse caso não há buraco a abrir: bastaria duplicar Q e mover para a cópia nova os elementos entre QP e a borda (como no Canônico de borda), deixando o lado curto do novo elemento na borda.

7 Documentação × implementação

Esta seção confronta as três fontes de documentação — a apresentação de projeto, os comentários do código e o artigo da IBHM — com o que o código faz. A apresentação foi preparada *antes* da implementação, de modo que parte das diferenças é simplesmente desatualização; ainda assim, vale registrá-las, porque a apresentação é hoje o principal documento do algoritmo. Cada item tem uma etiqueta: **[Erro]** (afirma algo falso sobre o código), **[Imprecisão]** (incompleto ou ambíguo), **[Desatualizado]** e **[Menor]** (redação, digitação).

7.1 Apresentação de projeto (`Operators_Template`)

O que confere

A decomposição em quatro operadores, as regras de decisão do slide 13, o comportamento do Canônico (aresta colapsada no interior, vértice novo na borda), o tratamento das arestas do meio (slide 14) e o exemplo completo (slides 15–27) coincidem com a implementação. Em particular, os 24 operadores anunciados no exemplo são exatamente os escolhidos pelo código, e o resultado final tem os mesmos buracos (K: 4 lados, L: 3, M: 6) e a mesma aresta colapsada em F.

A1. [Desatualizado] Nomes: os slides usam *Expand Collapsed Edge* (`expandCollapsedEdge`) e *Hole* (`hole(he)`); no código são `EXPAND_EDGE`/`expandEdge` e `OPEN_HOLE`/`openHole`.

A2. [Imprecisão] Slide 6, condição do Split Element: "*shares an outcome half-edge*". O código testa duas hipóteses: um lado do elemento de interface que *sai* do vértice (x_1) ou que *chega* nele (x_3), conforme o elemento compartilhado contenha *he* ou *b* (Seção 4.2). O slide 7 fala em "*income/outcome*", que é o correto. Também vale explicitar que a hipótese depende da posição da aresta nova em relação ao elemento de interface, não da half-edge escolhida pelo usuário.

A3. [Imprecisão] Slide 7: "*This operator replaces the algorithm that rotates around a vertex*". Na implementação os dois coexistem, e o Split Element é um atalho $O(1)$ do Expand Edge que produz a mesma topologia (Seção 4.5). Seria mais preciso dizer que ele *evita* a rotação quando o elemento compartilhado existe.

A4. [Desatualizado] Slide 9, Expand: "*We need to perform a traversal around the vertex starting from both element's side*". O código faz uma única rotação anti-horária, que começa na aresta colapsada e para na aresta nova. A travessia pelos dois lados existe, mas no Open Hole. A preocupação do

slide com o produto vetorial também não se aplica à solução adotada: a rotação é combinatória ($h \leftarrow \mathrm{O}[\mathrm{prev}(h)]$) e não usa geometria.

A5. **[Imprecisão]** Slides 11–12, Hole: "*the edge in the interface element is also duplicated for the new two vertices*" e "*both edge's vertices should be handled as in Expand*". O código não duplica aresta alguma: ele descola os lados curtos de dois elementos de interface que estavam colados (O ← BORDER), move um canto de um deles para o vértice novo, e usa duas travessias que param em elementos de interface — um procedimento diferente do Expand.

A6. **[Imprecisão]** Slide 13, regra 3 ("*part of an interface element, but not duplicated: this condition will open a hole*"): não distingue vértices internos de vértices de borda. O código segue a regra ao pé da letra e aplica Open Hole também na borda, onde não há buraco a abrir e a execução falha (Seção 6.2). A especificação precisa de um caso para a borda.

A7. **[Imprecisão]** Slide 5: "*the opposite half-edge for these edges is not defined*". No código ela é definida, com o valor sentinela COLLAPSED (−2), que as consultas reconhecem (`isBorder` pula essas arestas). Esse estado é uma extensão da estrutura de dados e merece ser documentado como tal.

A8. **[Menor]** Slide 2: "IBVHM" em vez de "IBHM".

A9. **[Menor]** O slide 27 tem o título "*Mesh Example – Edge 11*", igual ao do slide 26, mas mostra o estado após a aresta 12.

A10. **[Observação]** Slide 14: "*all the operators are designed to work on vertices. This property is desired in order to be easily translated to 3D*". Em 3D a duplicação acontece também em *arestas* (Seção 8.2); a afirmação vale como objetivo, mas é um ponto de risco.

7.2 Comentários do código

B1. **[Erro]** `openHole`, parâmetro `he`: "*This vertex should be part of a collapsed edge*". É o contrário: Open Hole é escolhido justamente quando o vértice *não* tem aresta colapsada (texto copiado de `expandEdge`). Ocorre nos dois construtores.

B2. **[Erro]** `CHE::addPoint` (e `IBHM::addPoint`): "*The function supposes that the required space is already allocated*". Mas `reserveSpaceForNodes` usa `resize`, e `addPoint` usa `push_back`: as coordenadas novas vão para depois do bloco reservado. Teste: numa malha com 4 vértices, `reserveSpaceForNodes(1)` seguido de `addPoint({5,7})` deixa o vetor com 12 valores (deveriam ser 10) e `coordinatesFromIndex(4)` devolve $(0, 0)$. Hoje passa despercebido porque todos os pontos novos são criados na origem.

B3. **[Imprecisão]** `heOpposite` e `_oppositeHalfEdge` documentam apenas o valor −1 (borda); falta COLLAPSED (−2).

B4. **[Imprecisão]** `computeNumberOfNewVertices`: "*n if it is in the border*". O código soma 1 *por aresta* que toca um vértice de borda, então um vértice de borda com duas arestas no mesmo lote é contado como 3 em vez de 2. Na IBHM, arestas que serão rejeitadas também são contadas. Em ambos os casos o efeito é só reservar memória a mais.

B5. **[Erro]** Documentação da classe `IBHMInterfaceElementBuilder`: "*There are only 4 operators to 3D*". A implementação é 2D; provavelmente a intenção era "apenas 4 operadores, pensados para serem estendidos a 3D".

B6. **[Imprecisão]** `reindexElementsCCW` não documenta a pré-condição de que a rotação não pode atravessar uma aresta colapsada: se $\mathrm{O}[\mathrm{prev}(h)] =$ COLLAPSED, o laço usa o sentinela como índice. Nos usos atuais isso não ocorre, mas é um invariante implícito.

B7. **[Menor]** `numberReservedElements`: na CHE o `@return` diz "*number of elements in the mesh*" (devolve a capacidade reservada); na IBHM devolve o número de *posições de half-edges* reservadas, não de elementos.

B8. **[Menor]** `openHole`: os `@todo Is it possible?` junto às asserções de borda. Sim, é possível (Seção 6.2).

B9. **[Menor]** Restos de cópia: cabeçalhos de `CHE.h` e `IBHM.h` com "*File: CHELevelOne.h*"; `IBHM::elementsList` cita "*The CHE class*"; `buildOppositesLinear` fala em "mapa do vértice para os elementos", mas guarda, em vetores, as half-edges que partem de cada vértice; "*the bother*" (border) na documentação do enum da IBHM.

B10. **[Menor]** Os dois construtores divergiram na documentação: o da IBHM tem a descrição da classe, dos operadores no `enum` e a observação "um construtor por malha"; o da CHE, que é a classe principal, não tem.

7.3 Artigo da IBHM

C1. **[Imprecisão no artigo]** Equação (1): $\text{next}_{he}(he, c) := Of[c] + \text{circ}(he + 1, n_v(c))$. Lida literalmente, com *he* sendo o índice global, a fórmula só funciona quando $Of[c]$ é múltiplo de $n_v(c)$, o que em geral não acontece em malhas híbridas. No próprio exemplo da Figura 4 do artigo, a face 2 (quadrilátero) tem $Of = 3$ e half-edges 4 a 7; a fórmula dá $\text{next}(4) = 3 + \text{circ}(5, 4) = 4$, quando deveria dar 5. A forma correta aplica o circ ao índice local: $Of[c] + \text{circ}(he - Of[c] + 1,\ n_v(c))$. A implementação usa a forma correta (`firstHE + (he - firstHE + 1) % n`).

C2. **[Imprecisão no artigo, menor]** Algoritmo 4 (estrela de um vértice): como está impresso, para um vértice de borda o laço executa $he \leftarrow \text{O}[\text{prev}(he)] = 0$ e insere $\text{cell}_{he}(0)$ antes de testar a condição de parada, e a célula de $VH[v]$ só é incluída quando o laço volta a ela (vértices internos). Ao que parece, a intenção é inserir primeiro a célula de $VH[v]$ e testar $\text{O}[\text{prev}(he)] = 0$ antes de avançar. O algoritmo não é usado pela implementação.

C3. **[Divergência]** A implementação não tem o vetor $VH[\,]$. Por isso `isBorder` precisa girar em torno do vértice ($O(\deg v)$, ou $O(\deg v \log n_c)$ na IBHM, em vez de $O(1)$), e é nessa rotação que mora o defeito D2 (Seção 7.5). Manter $VH[\,]$ atualizado pelos operadores é simples (cada operador sabe quais cantos mudou) e eliminaria as rotações de teste.

C4. **[Divergência]** Convenções diferentes (índices a partir de zero, -1 para borda, $n_c + 1$ offsets, estado COLLAPSED), resumidas na Tabela 1. Não são erros, mas devem ser registradas para quem comparar código e artigo.

C5. **[Observação]** O artigo aponta como limitação da IBHM o remalhamento dinâmico. A estratégia "contar → reservar → validar" da implementação é uma resposta a essa limitação e merece ser destacada em uma futura publicação.

7.4 Arquivos de dados

D1. **[Menor]** Em `malha2Hyb.txt` e `malha2.txt` o vértice 18 (T) tem coordenadas $(3, 4)$; nos slides ele está em $(4, 3)$. Com $(3, 4)$, o triângulo M–T–S e o quadrilátero M–N–T–S ficam degenerados (três vértices colineares em $x = 3$). A topologia não é afetada, mas qualquer visualização ou verificação geométrica será. As figuras deste relatório usam $(4, 3)$.

D2. **[Menor]** `malha.txt` está em outro formato (sem a coluna com o número de vértices por elemento) e contém duas malhas concatenadas; não é lido corretamente pelas funções atuais de `main.cpp`.

D3. **[Menor]** `meshIBHM.txt` reproduz a Figura 4 do artigo, mas o último triângulo começa em outro vértice, o que muda a numeração de suas half-edges em relação ao artigo.

7.5 Defeitos e limitações encontrados no código

A Tabela 6 reúne os problemas do código encontrados durante a análise e confirmados com o programa de teste, com uma sugestão de correção. Nenhuma correção foi aplicada ao código.

ID	Grav.	Onde	Defeito e sugestão
D1	alta	`openHole`	Vértice de borda já aberto recebendo nova aresta: asserção (debug) ou escrita fora do vetor (release). *Sugestão:* permitir que os dois laços parem na borda; se o laço anti-horário parar na borda, não há elemento a mover; no laço horário, testar $O[h]$ = BORDER *antes* de aplicar next. Nos vértices de borda os lados curtos já são de borda, então nada mais muda.
D2	alta	`isBorder`	Laço infinito quando a half-edge inicial vem logo após uma aresta colapsada: o desvio "pula" exatamente a half-edge de parada. Acontece quando a entrada contém uma half-edge de elemento de interface (nos dois construtores; a IBHM só rejeita essas arestas *depois* da contagem). *Sugestão:* validar a entrada antes de contar e testar a parada também após o desvio.
D3	média	`insertInterface Elements` (CHE)	Não rejeita arestas de elementos de interface; inserir a mesma aresta duas vezes cola um segundo elemento ao primeiro. *Sugestão:* a mesma verificação da IBHM.
D4	média	`addPoint`, `reserveSpace ForNodes`	Coordenadas novas gravadas fora do lugar (B2). *Sugestão:* usar `reserve` ou escrever na posição do índice; atribuir às cópias a coordenada do vértice de origem.
D5	baixa	`computeNumber OfNewVertices`	Contagem em excesso na borda e de arestas rejeitadas (B4).
D6	baixa	`retrieveAvailable Vertex`	Compara x_1 e x_3 com BORDER, mas não com COLLAPSED; o sentinela pode ser passado a prev/next (na IBHM dispara uma asserção). Só ocorre com arestas de elementos de interface na entrada.
D7	baixa	`insertInterface Elements` (CHE)	`printf` inalcançável (depois de um `break`) no `switch` de A; a mensagem do `switch` de B imprime a em vez de `b`; `%d` com `unsigned`.
D8	manut.	builders	As classes CHE e IBHM são cópias (≈95% iguais) e já divergiram (D3, B10). Um *template* sobre o tipo de malha evitaria isso.
D9	manut.	build	`CMakeLists.txt` exige Qt, que o código atual não usa; `memcpy`/`memset` e `std::upper_bound` são usados sem `<cstring>`/`<algorithm>` (compila por inclusão transitiva).

Tabela 6: Defeitos encontrados no código, por gravidade.

8 Considerações finais e próximos passos

O protótipo 2D cumpre o que a apresentação propôs: com quatro operadores locais, escolhidos apenas pelo estado do vértice, insere elementos de interface em qualquer ordem, sobre malhas que já contêm elementos de interface, em estruturas compactas baseadas em índices e com memória reservada antecipadamente. A verificação experimental mostrou topologias válidas, contagem exata de vértices e independência da ordem em todas as configurações suportadas. Os pontos fracos estão bem localizados: vértices de borda com mais de uma aresta e validação da entrada.

8.1 Um invariante que resume o algoritmo

Uma forma compacta de enunciar o que os operadores constroem, e que pode orientar tanto os testes quanto a extensão 3D, é a seguinte:

Invariante

Depois de inseridas as arestas de um conjunto S, cada vértice original v tem tantas cópias quantos são os componentes conexos da sua estrela depois de removidas as arestas de S. Cada componente usa uma única cópia.

Para um vértice interno com k arestas selecionadas, isso dá 1 cópia se $k \leq 1$ (a ponta de trinca, que fica colapsada) e k cópias se $k \geq 2$; para um vértice de borda, $k + 1$ cópias. O buraco que aparece com $k \geq 3$ é a consequência geométrica de ligar k cópias apenas por elementos de espessura nula. Como a partição em componentes não depende da ordem, qualquer algoritmo correto chega ao mesmo resultado, e é isso

que explica a independência da ordem observada. Os operadores são uma forma *incremental* e local de manter esse invariante.

8.2 Rumo ao 3D

Algumas observações, como hipóteses de trabalho para a próxima etapa:

- Em 3D a entrada são faces, e o elemento de interface é um prisma (faces triangulares) ou hexaedro (faces quadrilaterais) de espessura nula. A duplicação deixa de acontecer só nos vértices: as *arestas* das faces selecionadas também precisam ser duplicadas. Uma aresta compartilhada por k faces selecionadas se comporta como um vértice 2D com k arestas selecionadas, e girar em torno de uma aresta em 3D (de célula em célula pelas half-faces opostas) é o análogo exato de girar em torno de um vértice em 2D. Os operadores 2D devem reaparecer, quase sem mudanças, como operadores *de aresta*.
- A estrela de um vértice em 3D não é mais um ciclo, e sim uma superfície fechada (ou com borda) de células. Dividir essa estrela exige uma busca em largura pelas células que não atravessa faces selecionadas, e não uma rotação. O invariante acima continua valendo e dá o critério: uma cópia por componente.
- O estado colapsado tem análogos em 3D (faces laterais degeneradas do prisma), e o buraco das junções vira "tubos" ao longo das arestas de junção.
- O sentinela COLLAPSED, a pré-contagem e a estratégia contar/reservar/validar se transportam diretamente para a 3D-IBHM.

8.3 Próximos passos sugeridos

1. **Corrigir D1 e D2** (Tabela 6) e validar a entrada antes da pré-contagem (D3, D6).
2. **Transformar o programa de teste em testes automatizados** do repositório, com as verificações V1–V6 da Seção 6, o exemplo dos slides e casos aleatórios com ordens permutadas.
3. **Geometria e transferência de estado**: atribuir às cópias as coordenadas do vértice de origem e expor o mapa "vértice novo → vértice de origem". Numa análise adaptativa, os campos nodais (deslocamentos, poropressões etc.) dos vértices novos precisam ser copiados do vértice de origem; o construtor conhece essa informação no momento da criação.
4. **Unificar** os construtores CHE e IBHM em um *template* e, eventualmente, os operadores em uma primitiva comum de "divisão de leque" (Seção 4.5), mantendo o Split Element como otimização.
5. **Manter o vetor** $VH[\,]$ da IBHM, eliminando as rotações de `isBorder`.
6. **Documentar** o algoritmo a partir deste relatório: atualizar a apresentação (A1–A10) e os comentários (B1–B10).
7. **Integração com o GeMA** e, em seguida, a extensão 3D.

Referências

[1] M. Lage, L. F. Martha, J. P. Moitinho de Almeida, H. Lopes. *IBHM: index-based data structures for 2D and 3D hybrid meshes*. Engineering with Computers 33:727–744, 2017. DOI 10.1007/s00366-015-0395-0.

[2] M. Lage, T. Lewiner, H. Lopes, L. Velho. *CHE: a scalable topological data structure for triangular meshes*. Relatório técnico, PUC-Rio, 2005.

[3] M. Lage, T. Lewiner, H. Lopes, L. Velho. *CHF: a scalable topological data structure for tetrahedral meshes*. SIBGRAPI 2005, pp. 349–356.

[4] M. Mäntylä. *An Introduction to Solid Modeling*. Computer Science Press, 1988.

[5] W. Celes, G. H. Paulino, R. Espinha. *A compact adjacency-based topological data structure for finite element mesh representation*. Int. J. Numer. Methods Eng. 64:1529–1556, 2005.

[6] G. H. Paulino, W. Celes, R. Espinha, Z. Zhang. *A general topology-based framework for adaptive insertion of cohesive elements in finite element meshes*. Engineering with Computers 24:59–78, 2008.

[7] J. Coelho. *Adaptive Interface Elements Generation – 2D Case*. Apresentação (Tecgraf/PUC-Rio), versão de 25/02/2026. Arquivos `doc/ppt/Operators_Template.{pptx,pdf}`.

A Reprodução das figuras e dos testes

Os arquivos ficam em `doc/report/`:

- `tools/driver.cpp`: programa de teste. Compila os fontes originais de `core/` sem alterações (para ler o estado privado dos construtores, usa `#define private public` *apenas* nesta unidade de compilação), insere as arestas uma a uma, aplica as verificações V1–V6 e grava todos os estados intermediários em JSON.
- `tools/gen.py`: monta as malhas pequenas, chama o `driver` e gera as figuras TikZ em `figs/`.
- `tools/stress.py` e `tools/compare.py`: teste com malhas aleatórias e comparação de topologias.
- `relatorio.tex`, `styles.tex` e `secoes/`: o texto.

Comandos (a partir da raiz do repositório; `$W` é um diretório de trabalho qualquer):

```
c++ -std=c++17 -Icore/TopologyDataStructures -Icore/InterfaceElementsBuilder \
   doc/report/tools/driver.cpp core/TopologyDataStructures/*.cpp \
   core/InterfaceElementsBuilder/*.cpp -o $W/driver
python3 doc/report/tools/gen.py $W/driver doc/report/figs $W/work
python3 doc/report/tools/stress.py $W/driver 7 200 $W/stress # 200 casos, semente 7
python3 doc/report/tools/stress.py $W/driver 11 400 $W/stress # 400 casos, semente 11
cd doc/report && tectonic -X compile relatorio.tex # ou xelatex
```

Exemplo de saída do `driver` para o exemplo dos slides:

```
step 1 edge he=31 (10-15) A:CANONICAL B:CANONICAL new=1 predicted=1 ok
step 2 edge he=34 (10-11) A:SPLIT_ELEMENT B:CANONICAL new=1 predicted=1 ok
...
step 12 edge he=45 (32-18) A:OPEN_HOLE B:CANONICAL new=2 predicted=2 ok
TOTAL ERRORS: 0
```

As posições dos vértices nas figuras são calculadas apenas para visualização: cada cópia de um vértice é deslocada de uma pequena distância na direção média dos centróides dos elementos contínuos que a usam, e os cantos de um elemento de interface sobre uma aresta colapsada são afastados na direção do elemento vizinho, para que a aresta de comprimento zero fique visível.